

Customer Relationship Management (CRM) System

System Design & Specification Document

Prepared by: Amazi

Prepared for: Internship / Academic Project Review

Technology stack: ASP.NET Core Web API, Entity Framework Core, MS SQL Server

Date: June 2026

1. Introduction

1.1 Purpose

This document specifies the design of a Customer Relationship Management (CRM) system intended to help a sales organization track customers, manage sales opportunities, log interactions, and follow up on tasks. The system supports both individual (B2C) customers and business (B2B) clients within a single unified data model.

The system is built around fifteen entities: Users, Companies, Leads, Customers, Opportunities, Activities, Tasks, Tags, StageHistory, Attachments, Notifications, AuditLog, Products, and OpportunityLineItems.

1.2 Scope

The system covers seven functional areas:

- Lead management — capturing and qualifying inbound enquiries before converting them to contacts
- Contact management — storing and organizing customer and company records
- Sales pipeline management — tracking opportunities through configurable stages, including itemized quote lines
- Activity logging — recording calls, emails, meetings, and notes against customers and deals
- Task management and reporting — follow-up reminders, notifications, and pipeline analytics
- Document and file management — attachments tied to customers and opportunities
- System auditability — a general-purpose change log beyond pipeline stage transitions

1.3 Definitions

Term	Definition
Customer / Contact	An individual person tracked in the system, whether a private individual (B2C) or an employee of a client company (B2B).
Company / Account	A business entity that one or more customers may belong to. Used only for B2B relationships.
Opportunity	A potential sale or deal associated with a customer, tracked through a pipeline of stages until it is won or lost.
Activity	A logged interaction — call, email, meeting, or note — associated with a customer and optionally an opportunity.
Task	A scheduled follow-up action with a due date, assigned to a staff member.

Pipeline	The ordered set of stages an opportunity passes through, from creation to close.
Rep	Short for sales representative; a staff user who owns customers and opportunities.
Attachment	A file (document, image, PDF) linked to a Customer or Opportunity record.
Notification	A system-generated alert (in-app, and optionally email) tied to a Task or Opportunity event.
Audit Log	A general-purpose record of who changed what field on which entity, and when.
Product / Line Item	A priced item or service that can be added to an Opportunity to build up its estimated value from itemized parts.

2. System Overview

2.1 Core Concept

Sales representatives manage a portfolio of customers, which may be individuals or contacts at client companies. Each customer can have one or more open opportunities representing potential deals, each optionally broken into priced line items. Every interaction with a customer is logged as an activity, building a chronological history. Staff schedule tasks to ensure timely follow-up and receive notifications as due dates approach. Files relevant to a deal or contact are attached directly to the record. Managers monitor pipeline health through aggregate reporting, and all significant data changes are traceable through a general audit log.

2.2 User Roles

Role	Description	Typical permissions
Admin	System-wide administrator	Full access to all data, user management, system configuration, pipeline/tag configuration
Manager	Sales team lead	View and reassign all records owned by their team; full reporting access
Sales Rep	Front-line salesperson	Full access to their own customers, opportunities, activities, and tasks only

2.3 B2C / B2B Handling

The system treats individual and business customers with a single Customer entity rather than two parallel models. A customer record has an optional link to a Company record: if `CompanyId` is null, the customer is a B2C individual; if set, the customer is a contact belonging to that company (B2B). This avoids duplicating contact-management logic for two customer types while still allowing company-level rollups, such as total pipeline value across all contacts at a given company.

3. Database Design

3.1 Entity-Relationship Overview

The schema consists of fifteen entities: Users, Companies, Customers, Leads, Opportunities, Activities, Tasks, Tags, StageHistory, Attachments, Notifications, AuditLog, Products, and OpportunityLineItems. Users sit at the center as the owning party for customers, companies, and opportunities. Opportunities form the pipeline core, linking outward to activities, tasks, line items, and stage history.

Entity relationship summary:

- Users (1) — own (N) Customers, Companies, Opportunities
- Companies (1) — employ (N) Customers
- Customers (1) — have (N) Opportunities, Activities, Tasks, Attachments
- Customers (N) — tagged with (N) Tags, via CustomerTags
- Opportunities (1) — have (N) Activities, Tasks, StageHistory entries, Attachments, OpportunityLineItems
- Products (1) — appear on (N) OpportunityLineItems
- Users (1) — receive (N) Notifications
- Any entity (1) — generates (N) AuditLog entries

3.2 Entity Definitions — Core Entities

3.2.1 Users

Field	Type	Notes
UserId	int, PK	Identity column
Name	nvarchar(100)	Required
Email	nvarchar(255)	Required, unique
PasswordHash	nvarchar(255)	Required, never stored in plain text

Role	nvarchar(20)	Admin / Manager / SalesRep
CreatedAt	datetime2	Default: current timestamp

3.2.2 Companies

Field	Type	Notes
CompanyId	int, PK	Identity column
Name	nvarchar(150)	Required
Industry	nvarchar(100)	Optional
Website	nvarchar(255)	Optional
Address	nvarchar(255)	Optional
AssignedRepId	int, FK → Users	Nullable; the rep who owns the account

3.2.3 Customers

Field	Type	Notes
CustomerId	int, PK	Identity column
FirstName	nvarchar(100)	Required
LastName	nvarchar(100)	Required
Email	nvarchar(255)	Required
Phone	nvarchar(30)	Optional
CompanyId	int, FK → Companies	Nullable — null means individual (B2C)
Source	nvarchar(50)	e.g. Referral, Website, Advertisement, ColdCall

AssignedRepId	int, FK → Users	Required — owning rep
CreatedAt	datetime2	Default: current timestamp

3.2.4 Leads

A Lead represents an unqualified enquiry or prospect that has not yet been verified as a genuine sales opportunity. Keeping Leads separate from Customers prevents unverified contacts from polluting the main contact database until a rep has qualified them. When a Lead is qualified, it is converted into a Customer (and optionally a Company and an initial Opportunity) using a single conversion endpoint; the original Lead record is then marked as converted rather than deleted, preserving the source record for reporting.

Field	Type	Notes
LeadId	int, PK	Identity column
FirstName	nvarchar(100)	Required
LastName	nvarchar(100)	Required
Email	nvarchar(255)	Optional
Phone	nvarchar(30)	Optional
CompanyName	nvarchar(150)	Free-text company name before a Company record exists
Source	nvarchar(50)	e.g. Website, Referral, Advertisement, ColdCall, TradeShow
Status	nvarchar(20)	New / Contacted / Qualified / Disqualified / Converted
AssignedRepId	int, FK → Users	Nullable — the rep who owns the lead
ConvertedCustomerId	int, FK → Customers	Nullable — set when Status = Converted; links to the resulting Customer
Notes	nvarchar(max)	Optional free-text qualifying notes

CreatedAt	datetime2	Default: current timestamp
-----------	-----------	----------------------------

3.2.5 Opportunities

Field	Type	Notes
OpportunityId	int, PK	Identity column
CustomerId	int, FK → Customers	Required
Title	nvarchar(150)	Required
Stage	nvarchar(30)	New / Contacted / Qualified / Proposal / Negotiation / Won / Lost
EstimatedValue	decimal(12,2)	Required; may be auto-derived from line items (see 3.3.2)
ExpectedCloseDate	date	Optional
ActualCloseDate	date	Nullable; set when Won or Lost
OwnerId	int, FK → Users	Required — owning rep

3.2.6 Activities

Field	Type	Notes
ActivityId	int, PK	Identity column
CustomerId	int, FK → Customers	Required
OpportunityId	int, FK Opportunities →	Nullable — set only if tied to a specific deal
Type	nvarchar(20)	Call / Email / Meeting / Note
Subject	nvarchar(150)	Required

Description	nvarchar(max)	Optional free text
ActivityDate	datetime2	Required
CreatedBy	int, FK → Users	Required

3.2.7 Tasks

Field	Type	Notes
TaskId	int, PK	Identity column
CustomerId	int, FK → Customers	Nullable
OpportunityId	int, FK Opportunities →	Nullable
Title	nvarchar(150)	Required
DueDate	date	Required
Status	nvarchar(20)	Pending / Completed
AssignedTo	int, FK → Users	Required

3.2.8 Tags and CustomerTags

Field	Type	Notes
Tags.TagId	int, PK	Identity column
Tags.Name	nvarchar(50)	Required, unique
CustomerTags.CustomerId	int, FK → Customers	Composite PK (with TagId)
CustomerTags.TagId	int, FK → Tags	Composite PK (with CustomerId)

3.2.9 StageHistory

Field	Type	Notes
HistoryId	int, PK	Identity column
OpportunityId	int, FK → Opportunities	Required
OldStage	nvarchar(30)	Nullable — null on first entry
NewStage	nvarchar(30)	Required
ChangedAt	datetime2	Default: current timestamp
ChangedBy	int, FK → Users	Required

3.3 Entity Definitions

3.3.1 Attachments

Allows files (proposals, signed contracts, ID scans, images) to be attached to a Customer, Company, or Opportunity, keeping all relevant documents in one place alongside the records they belong to.

Field	Type	Notes
AttachmentId	int, PK	Identity column
CustomerId	int, FK → Customers	Nullable
CompanyId	int, FK → Companies	Nullable
OpportunityId	int, FK → Opportunities	Nullable
FileName	nvarchar(255)	Original uploaded file name
FileUrl	nvarchar(500)	Storage location (e.g. blob storage path/URL)
FileSizeBytes	int	Required, used for storage quota checks

UploadedBy	int, FK → Users	Required
UploadedAt	datetime2	Default: current timestamp

Exactly one of CustomerId, CompanyId, or OpportunityId should be set, enforced at the application layer, matching the pattern already used for Tasks.

3.3.2 Products and OpportunityLineItems

Allows an Opportunity's value to be built from itemized, priced components rather than a single manually-entered number, giving reps the ability to produce detailed quotes inside the pipeline.

Field (Products)	Type	Notes
ProductId	int, PK	Identity column
Name	nvarchar(150)	Required
DefaultPrice	decimal(12,2)	Required
IsActive	bit	Default true; inactive products are hidden from new quotes

Field (OpportunityLineItems)	Type	Notes
LineItemId	int, PK	Identity column
OpportunityId	int, FK → Opportunities	Required
ProductId	int, FK → Products	Required
Quantity	int	Required, default 1
UnitPrice	decimal(12,2)	Copied from Product at time of add; editable per line

Opportunities.EstimatedValue becomes a calculated/cached sum of (Quantity × UnitPrice) across its line items when at least one line item exists, falling back to manual entry for simple deals with no itemized quote.

3.3.3 Notifications

Overdue tasks are surfaced proactively through a real delivery mechanism rather than requiring the rep to actively check the Tasks screen.

Field	Type	Notes
NotificationId	int, PK	Identity column
UserId	int, FK → Users	Recipient
Type	nvarchar(30)	TaskDue / TaskOverdue / OpportunityStalled / Mention
Message	nvarchar(255)	Required
RelatedTaskId	int, FK → Tasks	Nullable
RelatedOpportunityId	int, FK Opportunities →	Nullable
IsRead	bit	Default false
CreatedAt	datetime2	Default: current timestamp

3.3.4 AuditLog

AuditLog generalizes the same append-only pattern used by StageHistory to any field change on any core entity, providing complete traceability across all records in the system.

Field	Type	Notes
AuditLogId	int, PK	Identity column
EntityName	nvarchar(50)	e.g. Customer, Opportunity, Company
EntityId	int	Primary key value of the affected record

FieldName	nvarchar(100)	Name of the field that changed
OldValue	nvarchar(max)	Nullable — null on creation
NewValue	nvarchar(max)	Required
ChangedBy	int, FK → Users	Required
ChangedAt	datetime2	Default: current timestamp

Rows are inserted only, never updated or deleted — the same append-only discipline as StageHistory. StageHistory is retained as a specialized table for pipeline-velocity reporting performance, rather than being merged into AuditLog.

3.4 Design Notes

- Soft delete: Customer, Company, and Opportunity records carry an IsDeleted flag rather than being hard-deleted, so historical Activities and StageHistory remain intact even if a customer record is later removed from active views.
- Indexing: Customers.Email, Customers.AssignedRepId, Opportunities.Stage, Opportunities.CustomerId, AuditLog.EntityName + EntityId, and Notifications.UserId + IsRead are the most frequently filtered columns and should be indexed.
- Stage as string vs. enum table: Stage remains a constrained string for simplicity. A normalized PipelineStages lookup table is the planned extension for supporting multiple pipelines (Section 14).
- Cascade behavior: deleting a Customer does not cascade-delete Activities, StageHistory, Attachments, or AuditLog entries; foreign keys on those tables use SetNull or restrict, paired with the soft-delete approach above.
- Storage of files: Attachments stores a URL/path, not the file binary itself, in the relational database — actual file bytes belong in object storage (e.g. Azure Blob Storage), keeping the database lean.

4. UI/UX Design

The interface follows a conventional sales-tool layout: a persistent left navigation sidebar, a top bar with search and the logged-in user's identity, and a main content area that switches between list, board, and detail views. All screens are responsive but designed desktop-first, matching typical CRM usage patterns.

4.1 Navigation Structure

A left sidebar provides access to the main modules, always visible:

- Dashboard — landing page after login
- Leads
 - Lead List
 - New Lead

- Customers — contact list and detail views
- Companies — account list and detail views
- Pipeline — Kanban board of opportunities
- Tasks — “My Tasks” and team task views
- Reports — dashboards and analytics
- Global Search — accessible from the top bar on every screen
- Settings — pipeline, tag, and product configuration (visible to Admin and Manager)
- Admin — user management (visible only to Admin role)

4.2 Screen Descriptions — Core Screens`

4.2.1 Dashboard

The landing page after login. Shows the current user's open task count (with overdue tasks highlighted), a summary of their open pipeline value by stage, and a short list of recently logged activities. Managers and Admins see an additional team-wide summary panel above their personal one. The dashboard also surfaces unread Notifications as a bell icon with a dropdown panel.

4.2.2 Customer List

A searchable, filterable table of customers showing name, company (if any), email, phone, assigned rep, source, and tags. Filters sit above the table: search box, company dropdown, source dropdown, tag multi-select, and assigned-rep dropdown (the latter hidden for Sales Reps, who only see their own). Clicking a row opens the Customer Detail view. The table supports multi-row selection for bulk actions (bulk tag, bulk reassign, bulk export to CSV).

4.2.3 Customer Detail

A two-column layout. The left column shows static contact information with inline editing. The right column shows a tabbed view: Activity Timeline, Opportunities, Tasks, an Attachments tab for uploaded files, and a History tab showing the AuditLog entries for that customer.

4.2.4 Company Detail

Similar structure to Customer Detail, but the right-hand panel lists all contacts belonging to the company, with an aggregate “total open pipeline value” figure shown at the top.

4.2.5 Pipeline Board

A Kanban-style board with one column per stage. Each opportunity appears as a card showing title, customer name, and estimated value. Cards are dragged between columns to change stage; a drag triggers the stage-change API call and writes a StageHistory row. A filter bar above the board allows filtering by owner and date range.

4.2.6 Opportunity Detail

Shown as a side panel or modal when a card is clicked. Displays title, value, stage, expected/actual close dates, and embeds the Activity Timeline and Tasks components. The panel includes a Line Items sub-

table for adding priced Products to build up the deal value, and an Attachments section for quotes or contracts.

4.2.7 My Tasks

A simple list grouped into Overdue, Due Today, and Upcoming sections. Each row shows task title, related customer/opportunity, and due date, with a one-click “mark complete” action. Managers can switch to a team view showing the same grouping per rep.

4.2.8 Reports

A dashboard of four panels: pipeline value by stage, win rate over a selectable date range, average time per stage, and lead source breakdown.

4.3 Screen Descriptions

4.3.1 Global Search

A single search box in the top bar, available from every screen, that queries across Customers, Companies, and Opportunities simultaneously and returns a grouped results dropdown. Reps can find any record without navigating to a specific module first.

4.3.2 Settings (Pipeline, Tags, Products)

A configuration area, visible to Admins and Managers, with three tabs: Pipeline Stages (reorder or rename stages without a code change), Tags (create/archive tags), and Products (manage the Products catalog used by Opportunity line items). This directly targets the feedback that the system had no configuration surface at all beyond user management — every other aspect of the system was hardcoded.

4.3.3 Task Calendar View

An alternative view of Tasks rendered as a monthly calendar rather than a flat list, letting a rep see workload distribution across a week or month at a glance. The list view (My Tasks) is retained as the default; the calendar is a toggle, not a replacement.

4.3.4 Leads List

A filterable table of Lead records showing name, company name (free-text), source, status, assigned rep, and creation date. Filters: status dropdown (New / Contacted / Qualified / Disqualified / Converted), source dropdown, assigned-rep dropdown (hidden for Sales Reps). A “Convert” action button on each qualified-status row opens a conversion modal where the rep confirms or adjusts name, company link, and whether to create an initial Opportunity; on confirm, the lead status is set to Converted and the user is navigated to the new Customer Detail screen. Converted leads are hidden from the default view but accessible via a “Show converted” toggle.

4.4 Interaction Notes

- Inline editing is preferred over separate edit pages for simple fields; a full form is only used for multi-step creation (e.g. New Opportunity).

- Destructive actions require a confirmation dialog, since deletes are soft-deletes but should not be triggered accidentally.
- Role-based UI hiding: Sales Reps never see the assigned-rep filter, Settings, or the Admin section; the UI hides controls a user has no permission to use, rather than showing them disabled.
- Toast notifications confirm successful saves, stage changes, and task completions without interrupting the current view.
- In-app Notifications use the same toast mechanism for real-time alerts, plus a persistent bell-icon panel for anything not yet acknowledged.

5. Functional Requirements

5.1 Lead Management

1. The system shall allow creation, viewing, editing, and disqualification of Lead records.
2. The system shall allow filtering leads by status, source, and assigned rep.
3. The system shall allow a Lead to be converted into a Customer (and optionally a Company and an initial Opportunity) via a single atomic conversion action.
4. Converted Lead records shall retain a reference to the resulting Customer record (ConvertedCustomerId) and remain visible in a “Show converted” filter view for source reporting.

5.2 Contact Management

5. The system shall allow creation, viewing, editing, and (soft) deletion of Customer records.
6. The system shall allow a Customer to optionally be linked to a Company.
7. The system shall allow searching and filtering customers by name, email, company, source, assigned rep, and tag.
8. The system shall allow tagging and untagging customers with one or more Tags.
9. (New) The system shall allow bulk tagging, bulk reassignment, and bulk export of selected customers from the Customer List.

5.3 Pipeline Management

10. The system shall allow creation of an Opportunity linked to a Customer, with an initial stage, estimated value, and expected close date.
11. The system shall allow moving an Opportunity between stages, recording each transition in StageHistory.
12. The system shall display opportunities grouped by stage (Kanban-style board view).
13. The system shall set ActualCloseDate automatically when an Opportunity is marked Won or Lost.
14. (New) The system shall allow adding one or more Product line items to an Opportunity, automatically recalculating EstimatedValue as the sum of line-item totals when line items are present.

5.4 Activity Logging

15. The system shall allow logging an Activity against a Customer, optionally linked to a specific Opportunity.
16. The system shall display a chronological activity timeline per customer, merging activities logged at both the customer and opportunity level.

5.5 Tasks and Follow-ups

17. The system shall allow creating a Task with a due date, assigned to a staff member, linked to a Customer and/or Opportunity.
18. The system shall allow marking a Task as Completed.
19. The system shall surface a “My Tasks” view showing each rep's pending and overdue tasks.
20. The system shall generate a Notification when a Task becomes due or overdue, and when an Opportunity has had no Activity logged for a configurable stale-deal threshold.
21. The system shall offer a calendar view as an alternative to the list view for Tasks.

5.6 Document and File Management

22. The system shall allow uploading file Attachments against a Customer, Company, or Opportunity.
23. The system shall display all Attachments for a record in a dedicated tab, with file name, size, uploader, and upload date.
24. The system shall allow deleting an Attachment, recording the deletion in AuditLog.

5.7 System Auditability

25. The system shall record an AuditLog entry whenever a tracked field on a Customer, Company, or Opportunity is changed, capturing old value, new value, who changed it, and when.
26. The system shall display a record's audit history in a “History” tab on its detail screen.

5.8 Reporting

27. The system shall calculate total open pipeline value, grouped by stage and by rep.
28. The system shall calculate win rate over a selectable date range.
29. The system shall report average time spent per pipeline stage, derived from StageHistory.
30. The system shall break down customers and won revenue by lead Source.

5.9 Access Control

31. Sales Reps shall only view and modify Customers, Opportunities, Activities, and Tasks they own.
32. Managers shall view and reassign all records owned by reps on their team.
33. Admins shall have unrestricted access, including user account management and system configuration.

6. API Design

The API follows REST conventions over ASP.NET Core Web API, returning JSON. All endpoints except authentication require a valid JWT bearer token in the Authorization: Bearer header. Role checks are applied via authorization policies registered at startup. Pagination: all list endpoints accept page (1-based, default 1) and pageSize (default 20, max 100) query parameters and return a standard envelope: { "data": [...], "page": 1, "pageSize": 20, "totalCount": 143, "totalPages": 8 }. Error responses: all 4xx and 5xx responses return a consistent JSON body: { "status": 400, "title": "Validation failed", "errors": { "Email": ["Email is required", "Email format is invalid"], "Title": ["Title must not exceed 150 characters"] } }. 400 = malformed request, 401 = missing/invalid token, 403 = insufficient permission, 404 = record not found, 422 = validation error, 429 = rate limit exceeded, 500 = unexpected server error.

6.1 Authentication

Method	Endpoint	Description
POST	/api/auth/login	Authenticate and receive a JWT
POST	/api/auth/refresh	Exchange a refresh token for a new access token

6.2 Leads

Method	Endpoint	Description
GET	/api/leads	List leads; filter by status, source, assignedRepId; paginated
GET	/api/leads/{id}	Get a single lead record
POST	/api/leads	Create a new lead record
PUT	/api/leads/{id}	Update lead details or status
POST	/api/leads/{id}/convert	Convert a qualified lead into a Customer (and optionally a Company and an initial Opportunity); sets Status = Converted and ConvertedCustomerId
DELETE	/api/leads/{id}	Disqualify / delete a lead (soft-delete; sets Status = Disqualified)

6.3 Customers

Method	Endpoint	Description
GET	/api/customers	List customers; supports filtering by search, companyId, repId, tag, source
GET	/api/customers/{id}	Get a single customer with company, tags, and assigned rep
POST	/api/customers	Create a new customer
PUT	/api/customers/{id}	Update customer details
DELETE	/api/customers/{id}	Soft-delete a customer
GET	/api/customers/{id}/timeline	Get merged activity timeline for a customer
POST	/api/customers/{id}/tags	Attach a tag to a customer
DELETE	/api/customers/{id}/tags/{tagId}	Remove a tag from a customer
POST	/api/customers/bulk	(New) Apply bulk tag/reassign action to a list of customer IDs
GET	/api/customers/{id}/audit	(New) Get AuditLog entries for a customer

6.4 Companies

Method	Endpoint	Description
GET	/api/companies	List companies
GET	/api/companies/{id}	Get a company with its contacts and aggregate pipeline value
POST	/api/companies	Create a company
PUT	/api/companies/{id}	Update a company

DELETE	/api/companies/{id}	Soft-delete a company
--------	---------------------	-----------------------

6.5 Opportunities

Method	Endpoint	Description
GET	/api/opportunities	List opportunities; filter by stage, ownerId, customerId
GET	/api/opportunities/board	Get opportunities grouped by stage for the Kanban view
GET	/api/opportunities/{id}	Get a single opportunity with related activities and tasks
POST	/api/opportunities	Create an opportunity
PUT	/api/opportunities/{id}	Update opportunity details
PATCH	/api/opportunities/{id}/stage	Move an opportunity to a new stage (writes StageHistory)
DELETE	/api/opportunities/{id}	Soft-delete an opportunity
GET	/api/opportunities/{id}/lineitems	(New) List line items for an opportunity
POST	/api/opportunities/{id}/lineitems	(New) Add a product line item; recalculates EstimatedValue
DELETE	/api/opportunities/{id}/lineitems/{lineItemId}	(New) Remove a line item; recalculates EstimatedValue

6.6 Activities and Tasks

Method	Endpoint	Description
GET	/api/activities	List activities; filter by customerId, opportunityId
POST	/api/activities	Log a new activity

GET	/api/tasks	List tasks; filter by assignedTo, status, dueBefore
GET	/api/tasks/my	Current user's pending and overdue tasks
GET	/api/tasks/calendar	(New) Tasks grouped by date for calendar rendering
POST	/api/tasks	Create a task
PATCH	/api/tasks/{id}/complete	Mark a task as completed

6.7 Attachments (New)

Method	Endpoint	Description
GET	/api/attachments	List attachments; filter by customerId, companyId, or opportunityId
POST	/api/attachments	Upload a new attachment
DELETE	/api/attachments/{id}	Delete an attachment (logged to AuditLog)

6.8 Notifications

Method	Endpoint	Description
GET	/api/notifications	List current user's notifications
PATCH	/api/notifications/{id}/read	Mark a notification as read
PATCH	/api/notifications/read-all	Mark all of the current user's notifications as read

6.9 Products and Settings

Method	Endpoint	Description
GET	/api/products	List products

POST	/api/products	Create a product
PUT	/api/products/{id}	Update a product
GET	/api/settings/pipeline-stages	List configured pipeline stages
PUT	/api/settings/pipeline-stages	Reorder/rename pipeline stages

6.10 Search (New)

Method	Endpoint	Description
GET	/api/search?q=	Cross-entity search across Customers, Companies, and Opportunities

6.11 Reporting

Method	Endpoint	Description
GET	/api/reports/pipeline-value	Open pipeline value grouped by stage and rep
GET	/api/reports/win-rate	Win rate over a given date range
GET	/api/reports/stage-velocity	Average time spent per stage
GET	/api/reports/lead-sources	Customer count and won revenue by source

7. Security & Authentication

Security is applied at every layer: transport, authentication, authorization, input validation, and rate limiting. This section specifies the exact mechanisms used so the implementation is unambiguous.

7.1 Transport Security

All traffic between client and server is served exclusively over HTTPS (TLS 1.2 minimum, TLS 1.3 preferred). HTTP requests are redirected to HTTPS at the application layer via ASP.NET Core's UseHttpsRedirection middleware. HSTS headers are set on all production responses with a minimum max-age of one year.

7.2 Password Storage

Passwords are hashed using BCrypt with a work factor of 12 (adjustable upward as hardware improves). Plain-text passwords are never logged, stored, or transmitted after the point of receipt. The Users.PasswordHash column stores only the BCrypt output string, never a reversible encoding. Password reset uses a time-limited, single-use token (see Section 7.3) rather than emailing the current password or a temporary plain-text substitute.

7.3 JWT Access and Refresh Tokens

Authentication is token-based using two-token flow:

- Access token: a signed JWT (HMAC-SHA256 or RS256) with a 15-minute expiry. Claims include sub (UserId), role (Admin / Manager / SalesRep), and iat / exp. The token is sent in the Authorization: Bearer header on every API call.
- Refresh token: a cryptographically random 256-bit token stored server-side (hashed, in a RefreshTokens table keyed to UserId) with a 7-day expiry. Presented to POST /api/auth/refresh to obtain a new access token without re-authentication. Refresh tokens are single-use: each call issues a new refresh token and invalidates the previous one (rotation).
- Logout: the server-side refresh token record is deleted or marked revoked, invalidating the session regardless of the access token's remaining expiry window.

JWT signing keys are stored in the hosting environment's secret store (not in appsettings.json or source control). Key rotation is handled by updating the secret and restarting the API; all existing access tokens expire within 15 minutes.

7.4 Password Reset Flow

- User submits their email to POST /api/auth/forgot-password.
- The server generates a cryptographically random token, stores its hash alongside an expiry (30 minutes) in a PasswordResetTokens table, and emails a reset link to the address on file. The response is always 200 OK regardless of whether the email exists, to prevent user enumeration.
- User submits the token and new password to POST /api/auth/reset-password. The server validates the token hash and expiry, updates PasswordHash, and deletes the reset token record.
- All active refresh tokens for that user are revoked on successful reset, forcing re-login on all devices.

7.5 Input Validation

All incoming request bodies are validated using ASP.NET Core's model validation (Data Annotations or FluentValidation) before reaching any service or database layer. Key rules:

- Required fields (Name, Email, Title, DueDate, etc.) are enforced server-side; a missing required field returns 422 Unprocessable Entity with a structured error body (see Section 6.1 API Conventions).
- Email fields are validated against a standard RFC 5322 pattern. Phone fields accept an optional “+” prefix and digits only, max 30 characters.
- String length limits match the database column sizes (e.g. Name $\leq$ 100 chars, Email $\leq$ 255 chars). Values exceeding limits return 422 before any database write attempt.
- Stage values are validated against the current PipelineStages configuration; an unknown stage name returns 422 rather than silently storing an invalid value.
- File uploads (Attachments) are validated for MIME type (allowlist: pdf, docx, xlsx, png, jpg) and file size (maximum 25 MB per file) before being written to object storage.
- Decimal values (EstimatedValue, UnitPrice, DefaultPrice) are validated as non-negative. Quantity is validated as a positive integer.

7.6 Rate Limiting

Rate limiting is applied using ASP.NET Core’s built-in RateLimiter middleware (available from .NET 7+). Two policies apply:

- Authentication endpoints (POST /api/auth/login, POST /api/auth/forgot-password): 10 requests per IP per minute, using a fixed-window limiter. Excess requests return 429 Too Many Requests with a Retry-After header.
- General API endpoints: 200 requests per authenticated user per minute, using a sliding-window limiter keyed on the UserId claim. This prevents a single account from flooding reporting or search endpoints.

7.7 Authorization Enforcement

Role-based authorization is enforced at the controller level via [Authorize(Policy = “...”)] attributes, not inline if-blocks. Three policies are registered at startup:

- RepOrAbove: any authenticated user (SalesRep, Manager, Admin). Applied to all standard CRUD endpoints.
- ManagerOrAbove: Manager or Admin. Applied to team-view endpoints, bulk reassignment, and reporting.
- AdminOnly: Admin role exclusively. Applied to user management and Settings configuration endpoints.

Resource-level ownership checks (a SalesRep may not access another rep’s customer even if they guess the ID) are enforced in the service layer by comparing the record’s AssignedRepId / OwnerId to the current user’s UserId claim. A mismatch returns 403 Forbidden, not 404, so the existence of the record is not inadvertently disclosed.

8. Key Workflows

8.1 New Lead Intake

34. A new enquiry arrives (via referral, website, ad, cold call, or trade show).
35. A rep creates a Lead record, setting Source, free-text CompanyName (if B2B), and Status = New.
36. The rep contacts the prospect and updates the Lead Status through New → Contacted → Qualified as qualification progresses.
37. Once qualified, the rep clicks Convert on the Lead List screen. The conversion endpoint creates a Customer record (and optionally a Company and an initial Opportunity), sets Lead.Status = Converted and Lead.ConvertedCustomerId, and navigates the rep to the new Customer Detail screen. The first StageHistory row is written at this point (OldStage = null, NewStage = New).

8.2 Moving an Opportunity Through the Pipeline

38. The rep logs Activities as the deal progresses.
39. As qualification occurs, the rep updates the Opportunity's stage via PATCH `/api/opportunities/{id}/stage`.
40. Each stage change writes a new StageHistory row.
41. If the deal closes, the stage is set to Won or Lost and ActualCloseDate is populated automatically.

8.3 Building a Quote with Line Items

42. During the Proposal stage, the rep opens the Opportunity Detail panel's Line Items section.
43. The rep adds one or more Products, each defaulting to its catalog price but editable per line.
44. The system recalculates EstimatedValue as the sum of (Quantity × UnitPrice) across all line items.
45. The itemized quote can be attached as a generated or uploaded PDF via the Attachments tab.

8.4 Follow-up Reminder Cycle

46. After a meaningful Activity, the rep creates a Task with a due date for the next step.
47. As the due date approaches, the system generates a Notification visible in the bell-icon panel.
48. The rep's "My Tasks" view (or Calendar view) surfaces tasks due today or overdue.
49. On completing the follow-up, the rep marks the Task Completed and typically logs a new Activity.

8.5 Manager Reassignment

50. A Manager views their team's customers and opportunities.
51. If a rep leaves or is overloaded, the Manager reassigns the AssignedRepId or OwnerId to another rep.
52. The reassignment is recorded as an AuditLog entry, capturing the old and new rep.
53. Historical Activities and StageHistory retain their original CreatedBy / ChangedBy values, preserving an accurate audit trail.

9. Development Phases

Phases are sequenced to produce a usable, demoable system as early as possible. The three new modules (Attachments, Notifications, Products/Line Items) are folded into the existing five-phase structure rather than appended as a separate late phase, so that breadth is visible throughout the build rather than bolted on at the end.

9.1 Phase 1 — Foundation

Goal: a working API and database with authentication.

- Set up the ASP.NET Core Web API project structure and EF Core DbContext.
- Create and apply the initial migration for all fifteen tables.
- Implement User entity, password hashing, and JWT-based login/refresh.
- Implement role-based authorization policies.

9.2 Phase 2 — Core Contact Management

Goal: Customers and Companies are fully manageable.

- CRUD endpoints for Leads, including the /convert endpoint and status transition logic.
- 54. CRUD endpoints for Customers and Companies, including the nullable CompanyId B2C/B2B link.
- Tag and CustomerTags endpoints.
- Attachment upload/list/delete endpoints, wired to the Customer and Company detail screens.
- Customer List and Customer Detail screens in the frontend, including bulk actions.
- Ownership scoping: Sales Reps see only their own customers; Managers/Admins see all.

9.3 Phase 3 — Pipeline

Goal: opportunities can be created and moved through stages, with itemized quotes.

- CRUD endpoints for Opportunities, plus the stage-change endpoint that writes StageHistory.
- Products and OpportunityLineItems endpoints; EstimatedValue recalculation logic.
- Pipeline Board (Kanban) screen with drag-to-change-stage.
- Opportunity Detail panel, including the Line Items sub-table.

9.4 Phase 4 — Activities, Tasks, and Notifications

Goal: the day-to-day interaction loop is complete, with proactive alerts.

- Activity logging endpoints and the merged timeline endpoint.
- Task endpoints, including the “my tasks” view and the new Calendar view.
- Notifications entity and endpoints; a background job (or on-request check) that generates TaskDue/TaskOverdue/OpportunityStalled notifications.

- Activity Timeline and Tasks tabs wired into Customer Detail and Opportunity Detail.

9.5 Phase 5 — Reporting, Auditability, and Polish

Goal: the system is presentation-ready and demonstrates full traceability.

- Reporting endpoints (pipeline value, win rate, stage velocity, lead sources).
- AuditLog write-through on Customer, Company, and Opportunity field updates; History tab on detail screens.
- Global Search endpoint and top-bar search UI.
- Settings screens for Pipeline Stages, Tags, and Products.
- Dashboard and Reports screens.
- Soft-delete behavior, validation messages, and UI polish pass.
- Seed data script for demoing with realistic sample customers, opportunities, attachments, and notifications.

9.6 Suggested Timeline

Phase	Focus	Estimated duration
1	Foundation (auth, DB for 14 tables, project setup)	1 weeks
2	Contact management + Attachments	1 weeks
3	Pipeline + Products/Line Items	1 weeks
4	Activities, Tasks + Notifications	1 weeks
5	Reporting, AuditLog, Search, Settings, polish	1 weeks

Total: approximately 5 weeks, reflecting the full breadth of the fourteen-entity schema, API surface, and UI screens described in this document.

10. Testing Plan

Testing is layered: fast unit tests for business logic, integration tests against a real (or in-memory) database, and manual end-to-end passes for the workflows that matter most to a working demo.

10.1 Unit Testing

Scope: service-layer logic that doesn't require a database — validation rules, stage-transition rules, role-based access checks, line-item total calculations, and report calculations.

- Tooling: xUnit, with Moq for mocking repository/service dependencies.

- Example cases: rejecting a stage change to an invalid stage name; calculating win rate correctly with zero closed deals; ensuring a Sales Rep cannot query another rep's customers; (new) recalculating EstimatedValue correctly when a line item is removed.

10.2 Integration Testing

Scope: EF Core queries and API endpoints against a real SQL Server (or SQLite/in-memory) test database.

- Tooling: xUnit with WebApplicationFactory for in-process API testing; a dedicated test database reset between test runs.
- Example cases: creating a Customer with a CompanyId correctly links to the Company; changing an Opportunity's stage writes exactly one new StageHistory row; soft-deleting a Customer hides it from list endpoints but leaves Activities intact; (new) uploading an Attachment creates a record with the correct parent link; (new) a field update on Customer writes an AuditLog row with correct old/new values.

10.3 Manual / End-to-End Testing

Workflow	Pass criteria
Login and role enforcement	A Sales Rep cannot view another rep's customers via the UI or by guessing a URL/ID
New lead intake	Creating a Customer and an initial Opportunity results in one StageHistory row and the deal appears on the pipeline board
Stage drag-and-drop	Dragging a card to a new column updates the stage, appends to StageHistory, and survives a page refresh
Activity logging	An activity logged against an Opportunity appears in both the Opportunity's and the parent Customer's timeline
Task completion	Marking a task complete removes it from Overdue/Due Today and no longer counts toward the dashboard's open task total
Reporting accuracy	Pipeline value and win rate shown in Reports match a manual count from the underlying data
Line item quote build	Adding two line items to an Opportunity updates EstimatedValue to their combined total; removing one updates it again
(New) Attachment upload	A file uploaded to a Customer appears in their Attachments tab with correct size, name, and uploader

Notification delivery	A task due today produces a notification visible in the bell-icon panel and disappears once marked read
Audit trail visibility	Changing a Customer's email shows the old and new value in the customer's History tab
Global search	Searching a partial customer name returns matching Customers, Companies, and Opportunities in one results list

10.4 Test Data

A seed script populates a representative dataset — a mix of B2C and B2B customers, opportunities spread across every stage with itemized line items on at least a few, a spread of overdue/due-today/upcoming tasks with corresponding notifications, sample attachments, and a populated AuditLog — so every screen has meaningful content during manual testing and demos.

11. Deployment Plan

As a solo academic/internship project, deployment favors low-cost and low-maintenance options over enterprise infrastructure, while still reflecting realistic practice.

11.1 Environments

Environment	Purpose	Database
Local development	Day-to-day coding and debugging	Local SQL Server (or SQL Server LocalDB)
Staging / demo	Stable build for instructor or portfolio review	Cloud-hosted SQL Server (e.g. Azure SQL free/basic tier)
Production (optional)	If the system is put into real use beyond the course	Managed SQL Server with backups enabled

11.2 Hosting

- API: deployed as an ASP.NET Core Web API on a free or low-cost tier (e.g. Azure App Service, or a container on Render/Railway).
- Database: Azure SQL Database (Basic/Serverless tier) or SQL Server in a managed container.
- File storage (New): Attachments require object storage — Azure Blob Storage (free/low-cost tier) rather than storing binaries in the relational database.
- Frontend: hosted as a static site (e.g. Azure Static Web Apps, Netlify, or Vercel) calling the API over HTTPS.

11.3 Configuration and Secrets

- Connection strings, JWT signing keys, and blob storage credentials are stored in environment variables / the hosting platform's secret store, never committed to source control.
- appsettings.json holds only non-sensitive defaults; appsettings.Production.json or environment variables override per environment.

11.4 Release Process

55. Run the full test suite (unit + integration) locally before pushing.
56. Push to the main branch, triggering a build via the hosting platform's CI.
57. Apply pending EF Core migrations against the target database as part of the deploy step.
58. Smoke-test the key workflows from Section 9.3 against the deployed environment, including the four new v2 workflows.

11.5 Backup and Rollback

- Automated daily backups on the hosted database where available; manual periodic export is an acceptable substitute on free tiers.
- Because deletes are soft (IsDeleted flag), most accidental data loss is recoverable without restoring from backup.
- Rollback is handled by redeploying the previous build artifact; database migrations are written to be additive where possible to avoid destructive rollbacks.

12. Non-Functional Requirements

Category	Requirement
Security	Passwords hashed (e.g. BCrypt); JWT-based authentication; role-based authorization on all endpoints; file uploads validated for type/size before storage
Performance	List endpoints support pagination; common filter columns are indexed; notification generation runs as a lightweight background check, not on every page load
Data integrity	Stage transitions only via the dedicated endpoint, ensuring StageHistory is never bypassed; line-item totals recalculated server-side, never trusted from client input
Auditability	StageHistory, AuditLog, and CreatedBy/ChangedBy fields provide a traceable history of key changes across all core entities, not just pipeline stage
Extensibility	Pipeline stages and lead sources are stored as constrained strings, allowing future move to configurable lookup tables; see Section 14 for the broader extensibility plan

Storage	File attachments are stored in object storage, not the relational database, keeping database size and backup time manageable as usage grows
---------	---

13. Future Enhancements

- Email integration: log emails automatically via a connected mailbox rather than manual activity entry.
- Multiple pipelines: support different stage sets for different product lines or sales motions, via a normalized PipelineStages table.
- Custom fields: allow admins to define additional fields on Customers or Opportunities without schema changes.
- Mobile app: native or PWA companion for reps in the field.
- Advanced reporting: cohort analysis, forecasting, and configurable custom report builder.

14. Extensibility and Multi-Tenancy Readiness

This section documents how the schema and architecture are deliberately shaped so that the larger items in Section 14 — particularly multi-tenancy — can be added later without a structural rewrite.

13.1 Multi-Tenancy Path

The schema is not multi-tenant today, by explicit scope decision (Section 1.2). However, every core entity (Users, Companies, Customers, Opportunities, Products, Tags) already has a single-owner-table shape that an OrganizationId column can be added to non-destructively:

59. Add a nullable OrganizationId to each core table, defaulting existing rows to a single seeded “default organization” during migration.
60. Add an Organizations table (OrganizationId, Name, CreatedAt).
61. Add OrganizationId to the JWT claims and a global EF Core query filter scoping every query to the current organization, mirroring the existing soft-delete query filter pattern already in use.
62. Once verified, make OrganizationId non-nullable.

13.2 Configurable Pipelines Path

Section 3.4 already notes Stage as a constrained string. The migration path to multiple configurable pipelines is: introduce a Pipelines table and a PipelineStages table (PipelineId, StageName, SortOrder), add a PipelineId to Opportunities, and validate Stage values against the selected pipeline's PipelineStages at the application layer instead of a hardcoded list.

13.3 Custom Fields Path

A common pattern for custom fields without per-tenant schema changes is an EAV-style CustomFieldValues table (EntityName, EntityId, FieldName, FieldValue), paired with a CustomFieldDefinitions table describing field name, type, and which entity it applies to.

13.4 Why This Section Matters

The intent of this section is to document that scope decisions — what was built versus deferred — were made deliberately. Naming the extensibility paths explicitly and showing the schema already accommodates them demonstrates a planned, staged approach to future growth.

15. Conclusion

This design specifies a complete CRM system built on fifteen entities, a full REST API, thirteen screens, and a five-phase development plan estimated 5 weeks. The schema covers lead capture, contact management, sales pipeline, activity logging, task tracking, file attachments, notifications, and full audit logging. A dedicated security section (Section 7), standard API conventions (Section 6.1), and a documented extensibility path for multi-tenancy and configurable pipelines round out a system that is buildable solo and phase-by-phase while demonstrating breadth across data modeling, API design, security, UI surface area, and forward-looking architecture.
